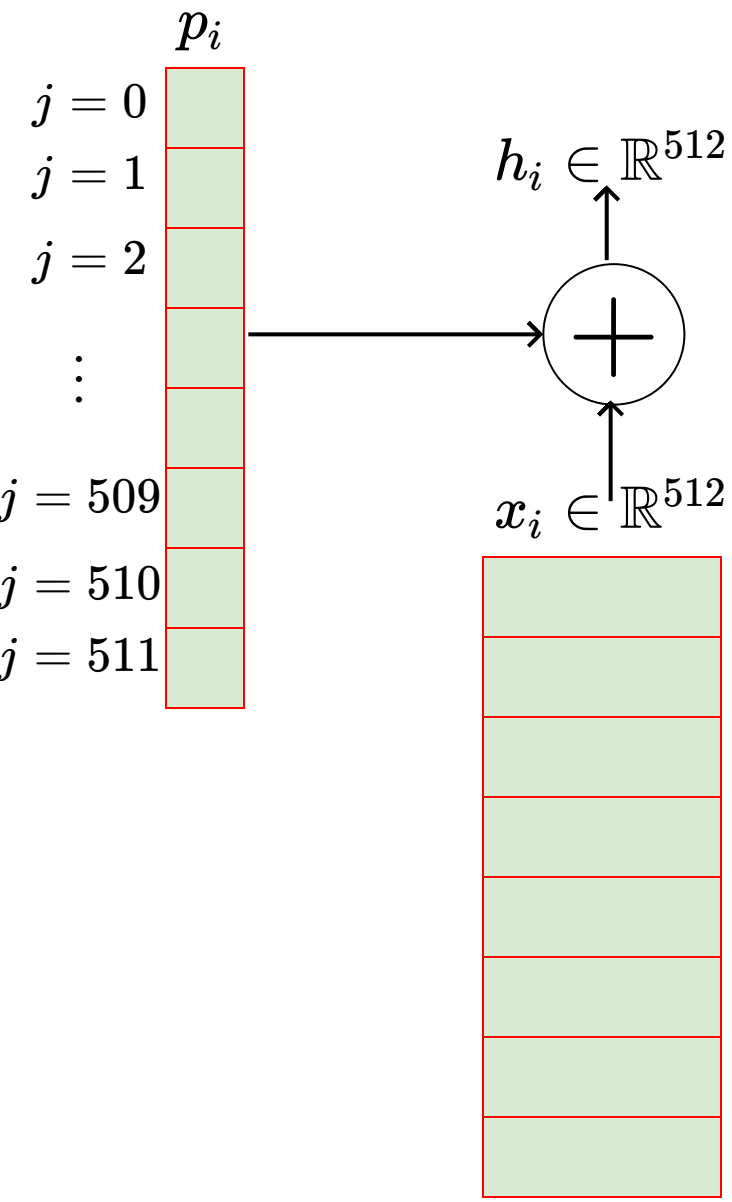
Introduction to Large Language Models

Position Encoding Methods and Length Generalization

Mitesh M. Khapra

AI4Bharat, Department of Computer Science and Engineering, IIT Madras

Absolute Positional Encoding



Positional encoding helps to capture the order of tokens in the input sequences

One approach is to encode the absolute positions

somewhere	something	incredible	is	waiting	to	be	known
0	1	2	3	4	5	6	7

we have done this by adding the position vector p_i with the word embedding x_i

$$h_i = x_i + p_i$$

the values for the position vector are fixed and come from

$$PE_{(i,j)} = \begin{cases} \sin\left(\frac{i}{10000^{2j/d_{\text{model}}}}\right), & j = 0, 2, \cdots \\ \cos\left(\frac{i}{10000^{2j/d_{\text{model}}}}\right), & j = 1, 3, \cdots \end{cases}$$

Question Everything

One approach is to encode the **absolute** positions

0 1 2 3

Why not **relative** positions?

-2 -1 0 1

by **adding** the position vector p_i with the **word embedding** x_i

$$x_i + p_i$$

Why not **multiply?**

$$x_i \odot f(p_i)$$

Why not add PE with the **attention score?**

$$q_i k_j^T + f(p_i, p_j)$$

the values for the position vector are **fixed** and come from

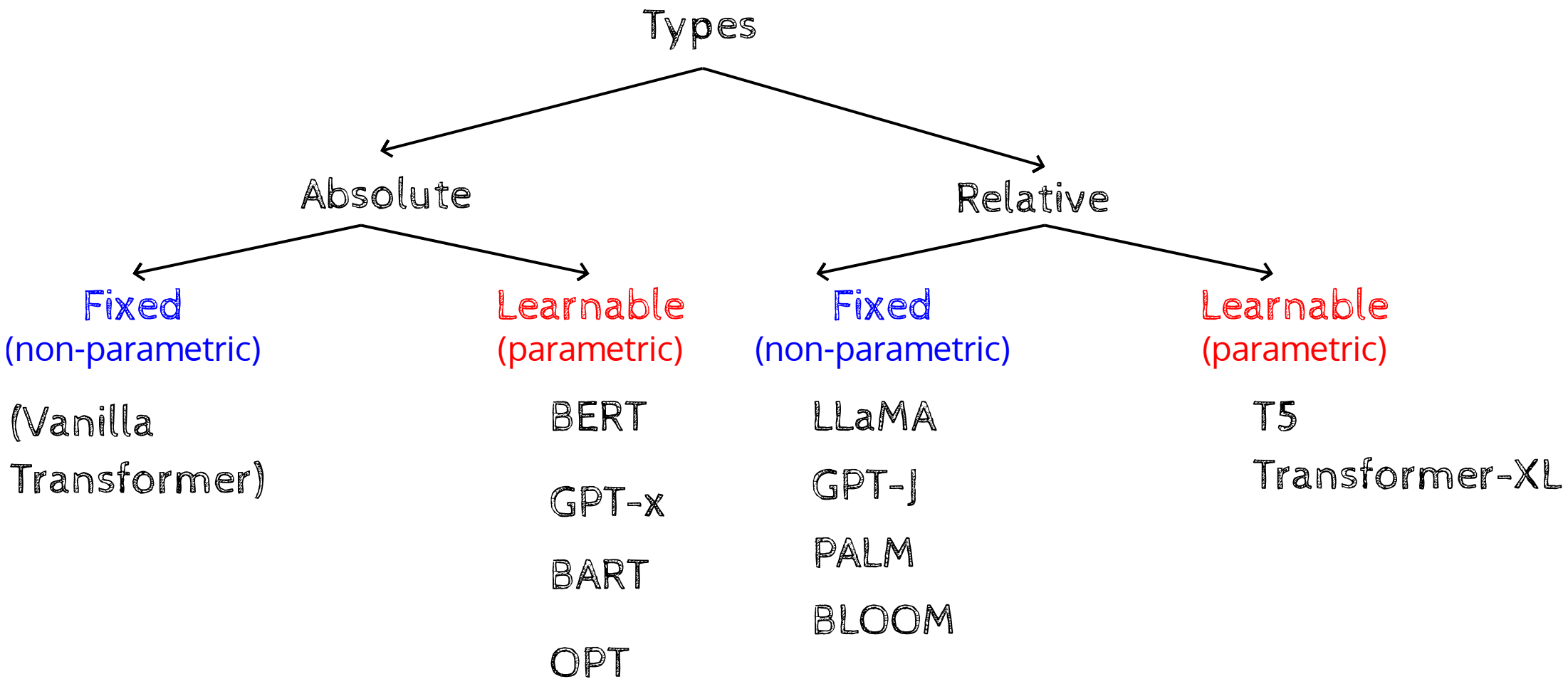
$$PE_{(i,j)} = (sin, cos)$$

Why not **learn** it?

$$PE(i,j) = (\theta, \nabla)$$

It is still an active area of research (alongside attention mechanisms)

We can broadly group existing methods under two categories

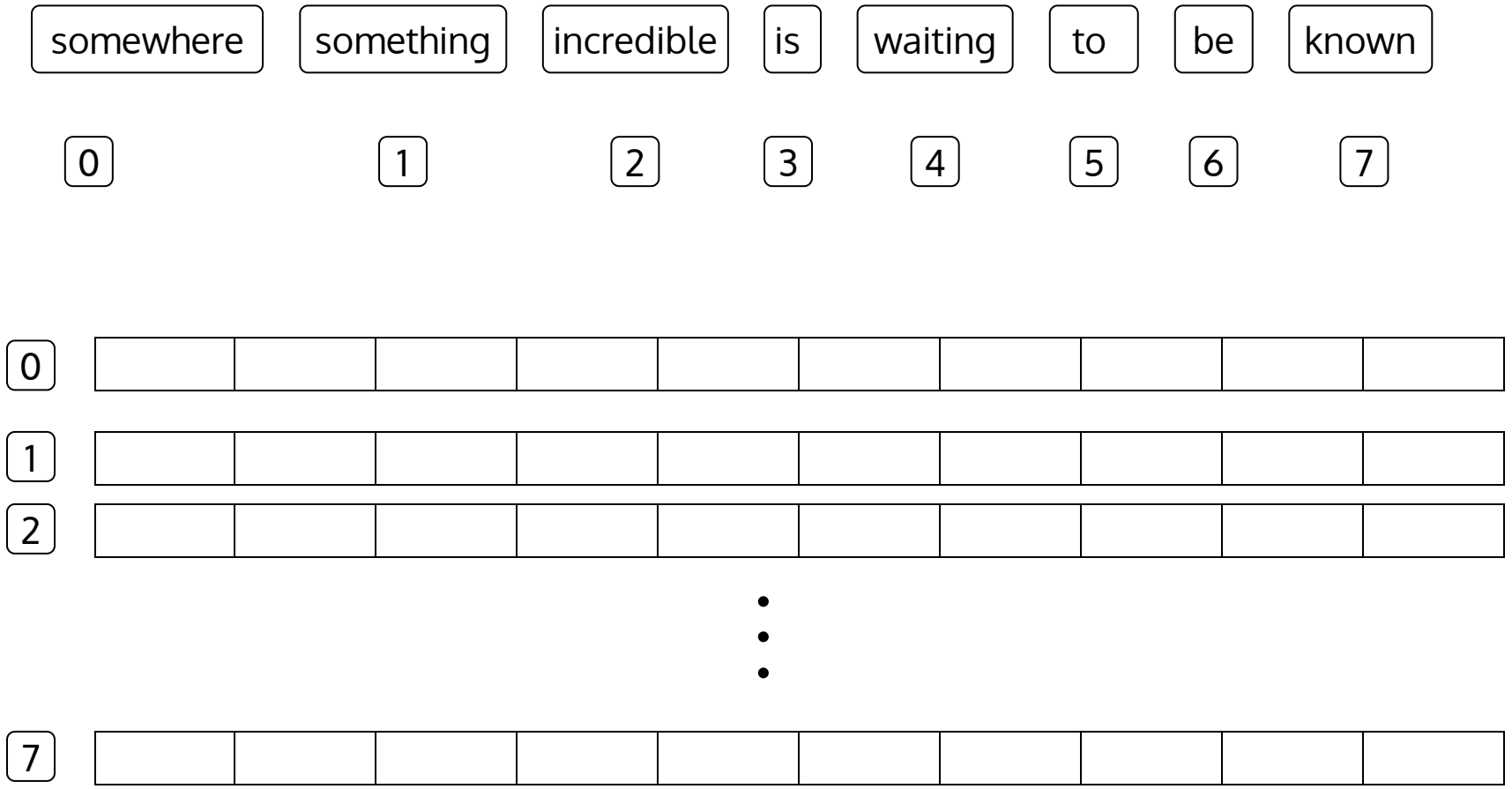


Module 1: Limitations of Absolute Position Encoding

Mitesh M. Khapra

AI4Bharat, Department of Computer Science and Engineering, IIT Madras

All prominent models like BERT, RoBERTa,GPT-X, BART and OPT use learned APE



It is hypothesized that APE does learn to attend to tokens by relative position (Why is this important? We will come back to this later)

Does it? Let's see

Suppose we take a pre-trained model (say, GPT-2) and use it for grammatical acceptance test (based on perplexity score).

Ideally, we expect the model prediction to be consistent for the given input sentence with (learned) absolute position encoding

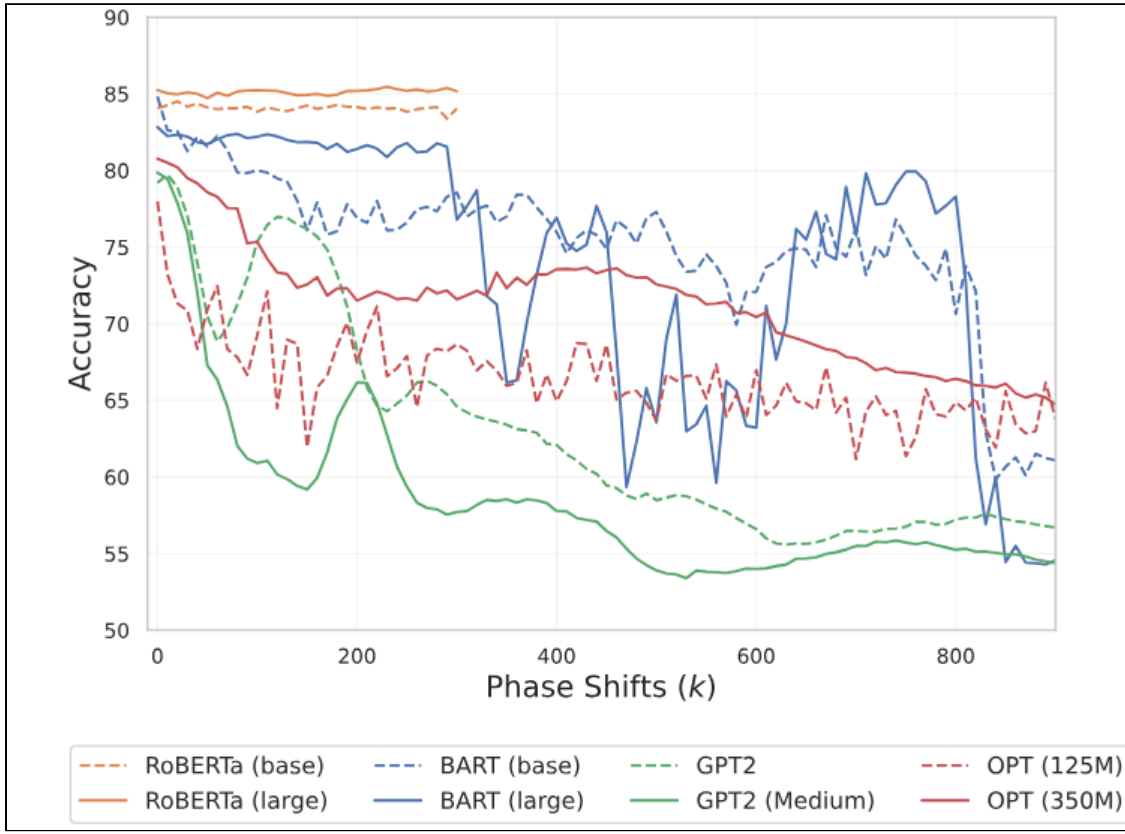
somewhere	something	incredible	is	waiting	to	be	known
0	1	2	3	4	5	6	7

What if we change the ***starting position*** of the input by shifting by k units to the right. (i.e., the embeddings of first token added to $(0 + k)$)

	somewhere	something	incredible	is	waiting	to	be	known
$k = 100$	100	101	102	103	104	105	106	107

If the model learned to attend to relative positions, then the prediction should not change. Therefore, the performance should not degrade.

However, it was observed that the performance degrades as k increases as shown in the figure [paper]



This motivated the need to explicitly encode relative positions

The other limitation of APE is its inability to extrapolate to a longer sequence beyond the context length of a model!

closely related to the above is the "length generalization" ability to extrapolate from shorter training sequences to longer test ones (within the context length of a model)

Let's take an example to understand these better

Suppose a GPT (uses APE) model was pre-trained with a context length T equals to 512.

However, we want the model to generate a story of 1024 tokens.

Well, we do not have the learned embeddings for the positions 513 to 1024. This would require us to re-train the model with $T = 1024$

However, re-training the same model with increased context length is computationally expensive.

Therefore, enabling the model to extrapolate to sequences longer than T is important

Now, consider the task of adding two integers using LLMs with a context length of 128

Say, all training examples demonstrate how to add two 4 digit integers

$$\begin{array}{r} 9999 \\ 0001 \\ \hline 10000 \end{array}$$

Can the model generalize the algorithm to add, say, two 12 digit integers?

Yes, the type of PE is a major factor in enabling this ability, but APE is not a good choice! [Paper]

Module 2 : Relative Position Encoding Methods

Mitesh M. Khapra

AI4Bharat, Department of Computer Science and Engineering, IIT Madras

We have only one **absolute position** for each token in the sequence

	somewhere	something	incredible	is	waiting	to	be	known
APE	0	1	2	3	4	5	6	7

However, in the case of Relative Position Encoding (RPE), we have T relative positions for each token in a sequence of length T

Therefore, RPE encodes pairwise relationships between tokens. (APE does not)

Let us understand this better

The relative position of the word "is" 0 with respect to the position of the word itself and differs with respect to the position of the words surrounding it

	somewhere	something	incredible	is	waiting	to	be	known
APE	0	1	2	3	4	5	6	7
RPE	-3	-2	-1	0	1	2	3	4

Therefore, the relative positions for the word "is" are (-3,-2,-1,0,1,2,3,4)

We have T relative positions for each token in a sequence of length T and, in general, $2T - 1$ embeddings to encode all relative positions (whereas APE requires only T embeddings)

Let us illustrate by displaying all relative positions for the above sequence

	somewhere	something	incredible	is	waiting	to	be	known
somewhere	0	1	2	3	4	5	6	7
something	-1	0	1	2	3	4	5	6
incredible	-2	-1	0	1	2	3	4	5
is	-3	-2	-1	0	1	2	3	4
waiting	-4	-3	-2	-1	0	1	2	3
to	-5	-4	-3	-2	-1	0	1	2
be	-6	-5	-4	-3	-2	-1	0	1
known	-7	-6	-5	-4	-3	-2	-1	0

We can view the pairwise relationship of tokens in the sequence as a labelled, directed, fully-connected graph

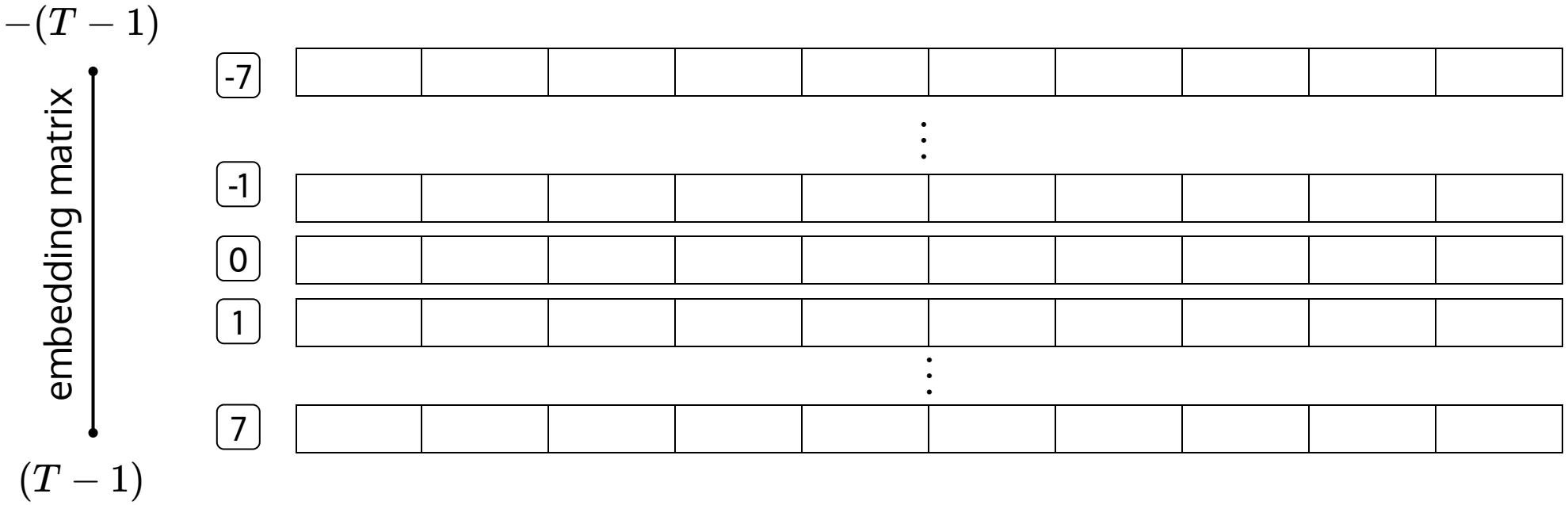
For example, a directed edge from the word "somewhere" to "something" has a label 1. Whereas, a directed edge from the word "something" to "somewhere" has a label -1.

With a naive implementation, for each token embedding x_i, we need a way to combine (say, addition) T relative position embeddings.

For $T = 8$

$$h_3 = x_3 + (p_{-3} + p_{-2} + \cdots + p_4)$$

$$h_i = x_i + \sum_{j=0}^{T-1} p_{j-i}$$



With naive implementation, for each token embedding x_i, we need a way to combine (say, addition) T relative position embeddings.

For $T = 8$

$$h_3 = x_3 + (p_{-3} + p_{-2} + \cdots + p_4)$$

$$h_i = x_i + \sum_{j=0}^{T-1} p_{j-i}$$

So, if $T = 1024$, then we need to do 1024 additions for each token in the input sequence!.

In general, it requires T^2 additions instead of T additions as in APE.

Can we do better?

The Idea

Let's look at the decomposition of the pre-attention computation

$$e_{ij} = q_i k_j^T$$

recall that,

$$q_i = h_i W_Q = (x_i + p_i) W_Q,$$
$$k_j = h_j W_K = (x_j + p_j) W_K$$

Therefore,

$$e_{ij} = (x_i + p_i) W_Q W_K^T (x_j + p_j)^T$$
$$= (x_i W_Q + p_i W_Q)(W_K^T x_j^T + W_K^T p_j^T)$$
$$= \boxed{x_i W_Q W_K^T x_j^T} + \boxed{x_i W_Q W_K^T p_j^T + p_i W_Q W_K^T x_j^T} + \boxed{p_i W_Q W_K^T p_j^T}$$

score for the given query and key without any position encoding

Correlation beween a word and position

Adding a constant that comes from position encoding

PE just adds a constant to the pre-attention score. Therefore, we can inject position information in the attention layer directly by adjusting the pre-attention score e_{ij}

The Idea

$$e_{ij} = q_i k_j^T$$

recall that, $q_i = h_i W_Q = (x_i + p_i) W_Q,$

$$k_j = h_j W_K = (x_j + p_j) W_K$$

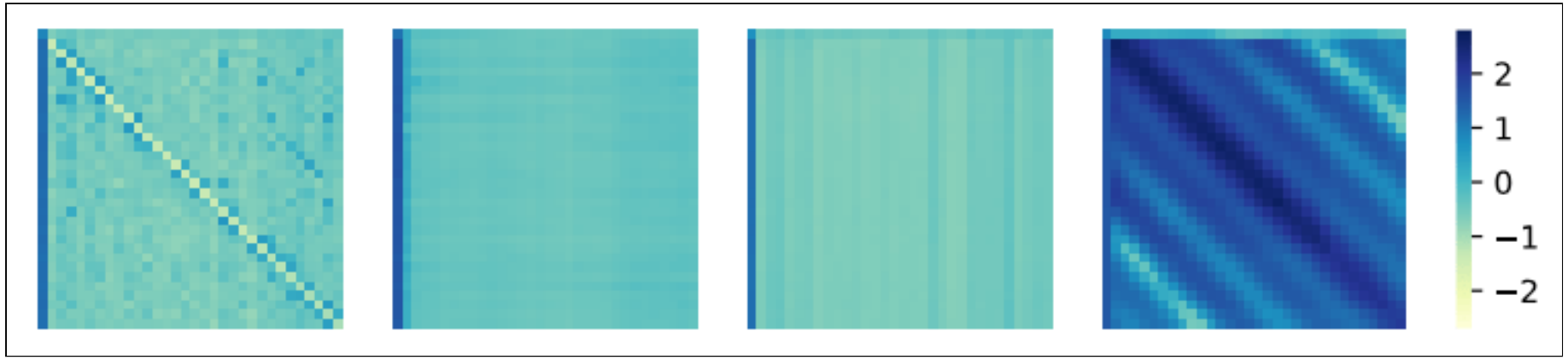
Therefore,

$$e_{ij} = (x_i + p_i) W_Q W_K^T (x_j + p_j)^T$$

$$= (x_i W_Q + p_i W_Q)(W_K^T x_j^T + W_K^T p_j^T)$$

$$= x_i W_Q W_K^T x_j^T + x_i W_Q W_K^T p_j^T + p_i W_Q W_K^T x_j^T + p_i W_Q W_K^T p_j^T$$

Adding APE to the input embeddings introduces positional artefact



Visualization of the above equation in BERT [paper]. From left: Correlation between word-to-word, word-to-position, position-to-word, position-to-position

The Idea

$$e_{ij} = q_i k_j^T$$

APE introduces positional artefact

recall that, $q_i = h_i W_Q = (x_i + p_i) W_Q,$

$$k_j = h_j W_K = (x_j + p_j) W_K$$

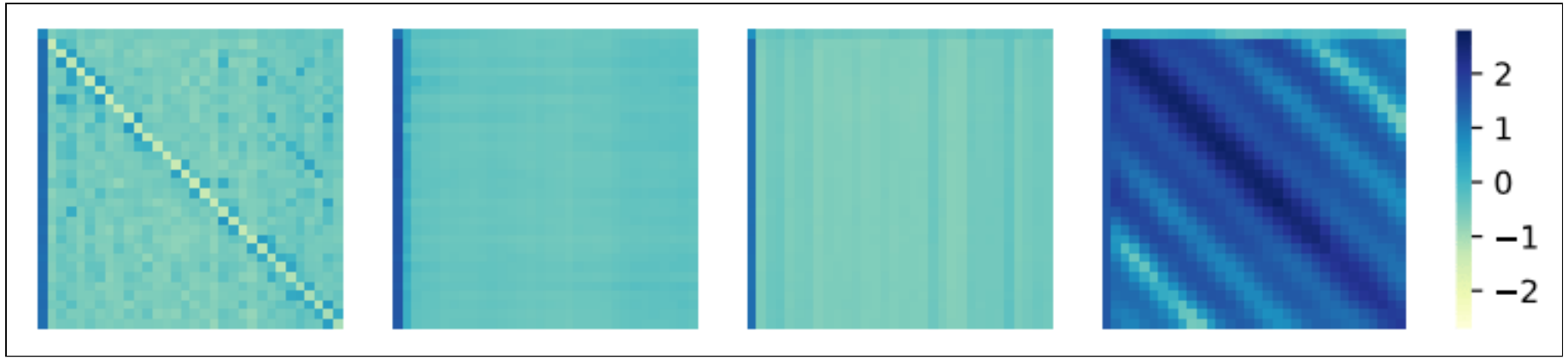
Therefore,

$$e_{ij} = (x_i + p_i) W_Q W_K^T (x_j + p_j)^T$$

$$= (x_i W_Q + p_i W_Q)(W_K^T x_j^T + W_K^T p_j^T)$$

$$= x_i W_Q W_K^T x_j^T + \boxed{x_i W_Q W_K^T p_j^T + p_i W_Q W_K^T x_j^T} + p_i W_Q W_K^T p_j^T$$

Uniformly distributed (that is, no correlation)



Visualization of the above equation in BERT [paper]. From left: Correlation between word-to-word, word-to-position, position-to-word, position-to-position

With naive implementation, for each token embedding x_i, we need to add T relative position embeddings

For $T = 8$

$$h_3 = x_3 + (p_{-3} + p_{-2} + \cdots + p_4)$$

$$h_i = x_i + \sum_{j=0}^{T} p_{j-i}$$

So, if $T = 1024$, then we need to do 1024 additions for each token in the input sequence!.

In general, it requires T^2 additions instead of T additions in APE.

Can we do better?

Yes, by modifying the pre-attention score e_{ij} directly

There are multiple ways of modifying the attention score !

Adding a constant to the attention score is the simplest approach!

$$e_{ij} = x_i W_Q (x_j W_K + p_{ij}^K)^T \qquad \text{where, } p_{ij}^K \in \mathbb{R}^d$$

p_{ij}^K is also called position bias (a bias b as in $xW + b$)

Note that in this formulation, the dimension of p_{ij} is d (head dimension), whereas in APE, it is d_{model}

Does this approach help the model to generalize to sequence lengths not seen during training?

No. Because the size of the position embedding matrix still depends on T !

Suppose the relative distance information is clipped beyond a certain distance k as follows

$$p_{ij}^K = w_{\text{clip}(j-i,k)}^K$$

$$\text{clip}(j-i,k) = \max(-k, \min(j-i,k))$$

Then, the size of position embedding matrix to $(2k+1)$, making it independent of T

It is also empirically observed that clipping the distance does not hurt the performance.

$$p_{ij}^{K} = w_{\text{clip(j-i,k)}}^{K}$$

$$\text{clip}(j-i,k) = \max(-k, \min(j-i,k))$$

$j = 5$

		somewhere	something	incredible	is	waiting	to	be	known
$i = 0$	somewhere	0	1	2	3	4	4	4	4

for $T = 8, k = 4$

$$p_{ij}^{K} = w_{\text{clip(j-i,k)}}^{K}$$

$$\text{clip}(j-i,k) = \max(-k, \min(j-i,k))$$

for $T = 8, k = 4$

	somewhere	something	incredible	is	waiting	to	be	known
somewhere	0	1	2	3	4	4	4	4
something	-1	0	1	2	3	4	4	4
incredible	-2	-1	0	1	2	3	4	4
is	-3	-2	-1	0	1	2	3	4
waiting	-4	-3	-2	-1	0	1	2	3
to	-4	-4	-3	-2	-1	0	1	2
be	-4	-4	-4	-3	-2	-1	0	1
known	-4	-4	-4	-4	-3	-2	-1	0

$$e_{ij} = x_i W_Q (x_j W_K + p_{ij}^K)^T = x_i W_Q W_K^T x_j^T + x_i W_Q (p_{ij}^K)^T$$

We can compute the attention score as usual

$$\alpha_{ij} = \mathrm{softmax}(e_{ij})$$

Finally, add the relative position information to value vectors

$$z_i = \sum_j \alpha_{ij} (x_j W_V + p_{ij}^V)$$

The position embeddings p_{ij}^K, p_{ij}^V are shared across heads (not across layers)

Replacing the APE in vanilla transformer architecture by RPE with $k = 16$ leads to better performance

Model	Position Information	EN-DE BLEU	EN-FR BLEU
Transformer (base)	Absolute Position Representations	26.5	38.2
Transformer (base)	Relative Position Representations	**26.8**	**38.7**
Transformer (big)	Absolute Position Representations	27.9	41.2
Transformer (big)	Relative Position Representations	**29.2**	**41.5**

Source: [paper]

RPE Variations

recall the decomposition

$$e_{ij} = x_i W_Q W_K^T x_j^T + x_i W_Q W_K^T p_j^T + p_i W_Q W_K^T x_j^T + p_i W_Q W_K^T p_j^T$$

Transformer-XL (recursivevly extend context)

$$e_{ij} = x_i W_Q W_K^T x_j^T + x_i W_Q R_{j-i}^T + u W_K^T x_j^T + v W_K^T R_{j-i}^T$$

Where,

R_{i-j}^T is the relative distance between the position i and j computed from sinusoidal function (non-learnable)

u, v are learnable vectors.

T5-bias

All that is required is adding a constant, why not just do that?

$$e_{ij} = x_i W_Q W_K^T x_j^T + r_{j-i}$$

Where,

$r_{i-j} \in \mathbb{R}$ are learnable **scalars**, shared across layers. Maximum relative distance is clipped to 128

It also greatly reduces number of learnable parameters when the model size is scaled

Rotary Position Embedding (RoPE)

Shaw $$e_{ij} = \boxed{x_i W_Q W_K^T x_j^T} + x_i W_Q (p_{ij}^K)^T$$

T-XL $$e_{ij} = \boxed{x_i W_Q W_K^T x_j^T} + x_i W_Q R_{j-i}^T + u W_K^T x_j^T + v W_K^T R_{j-i}^T$$

T5 $$e_{ij} = \boxed{x_i W_Q W_K^T x_j^T} + r_{j-i}$$

We can generalize this by letting,

$$f_q(x_i, i) = x_i W_Q$$

$$f_k(x_j, j) = x_j W_K$$

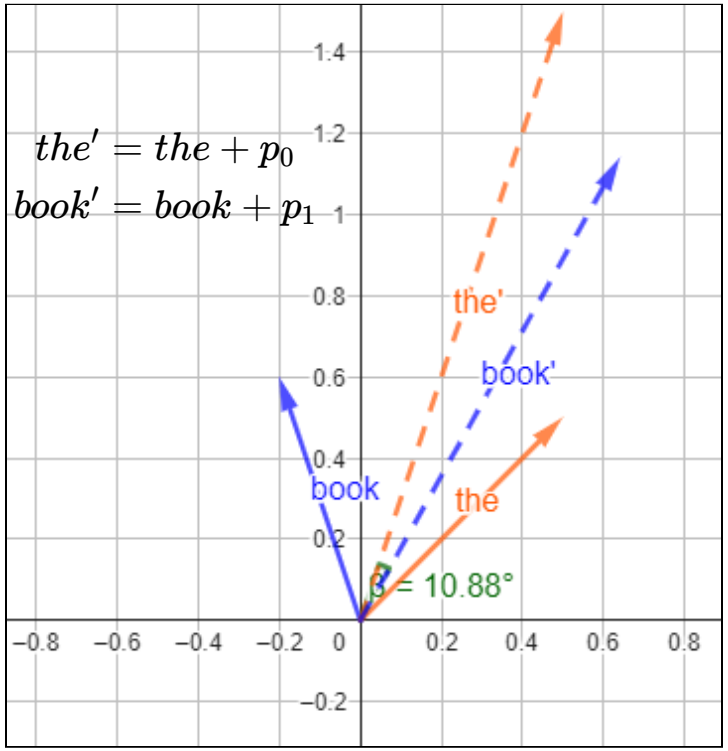
$$\langle f_q(x_i, i), f_k(x_j, j) \rangle = x_i W_Q W_K^T x_j^T$$

However, we want the inner product between tokens embeddings to encode the relative position information. That is we want it to be

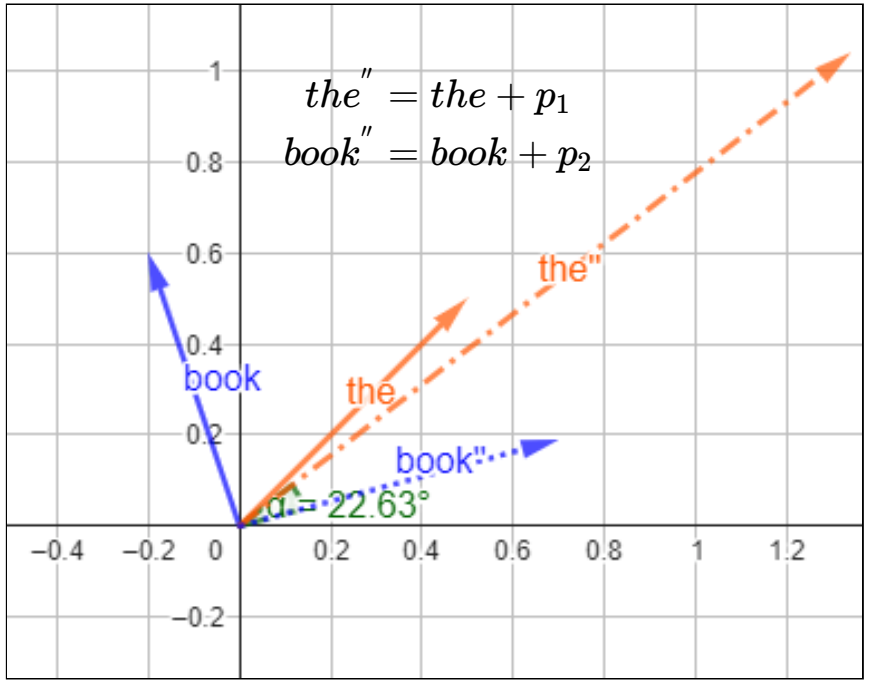
$$\langle f_q(x_i, i), f_k(x_j, j) \rangle = g(x_i, x_j, j - i)$$

Consider the following pair of words in a sentence

the book . . .



read the book . . .



APE

$$PE = \begin{array}{c} p_0 \rightarrow \\ \\ \\ p_3 \rightarrow \end{array} \begin{bmatrix} 0 & 1 \\ 0.84 & 0.54 \\ 0.9 & -0.41 \\ 0.14 & -0.98 \end{bmatrix}$$

$$q \cdot k = |q||k|cos(\theta)$$

Adding absolute position embedding changes the angle between the representation of words "the" and "book" based on where they appear in a sentence despite their relative distance being the same in both sentences

Can we preserve the angle between word representations so that the relative information can be encoded in angles?

The relative distance between the words "the" and "book" is 1 in all the sentences given below

the book . . .

$m = 0, n = 1$

read the book . . .

$m = 1, n = 2$

you read the book . . .

$m = 2, n = 3$

you must read the book . . .

$m = 3, n = 4$

he says you must read the book . . .

$m = 5, n = 6$

https://www.geogebra.org/material/iframe/id/tywaxmaw/width/700/height/500/border/888888/sfsb/true/smb/false/stb/false/stbh/false/ai/false/asb/false/sri/false/rc/false/ld/false/sdz/true/ctl/false

$$q \cdot k = |q||k|cos(\theta)$$

$$\langle f_q(x_i, i), f_k(x_j, j)\rangle = g(x_i, x_j, j - i)$$

Let us assume the following functional form for the left-hand side of the above equation (keeping rotation in mind)

$$f_q(x_m, m) = x_m W_Q e^{jm\theta}$$

$$f_k(x_n, n) = x_n W_K e^{jn\theta}$$

(replaced i, j by m, n to avoid confusion with the imaginary number)

Here, we multiply a (complex) constant to the affine-transformed word embedding

Recall that the inner product between two complex vectors $x, y \in \mathbb{C}^d$ is $x \cdot y^H$, where H is a Hermitian transpose

then

$$\begin{aligned}\langle f_q(x_m, m), f_k(x_n, n)\rangle &= x_m W_Q e^{jm\theta}(x_n W_K e^{jn\theta})^H \\ &= x_m W_Q (x_n W_K)^T e^{j(m-n)\theta}\end{aligned}$$

$$g(x_m, x_n, m - n) = Re[(x_m W_Q)(x_n W_K)^T e^{j(m-n)\theta}]$$

θ is a non-zero constant (why?).

$$\mathbf{A} = \begin{bmatrix} 1 & -2-i & 5 \\ 1+i & i & 4-2i \end{bmatrix}$$

We first transpose the matrix:

$$\mathbf{A}^{\mathrm{T}} = \begin{bmatrix} 1 & 1+i \\ -2-i & i \\ 5 & 4-2i \end{bmatrix}$$

Then we conjugate every entry of the matrix:

$$\mathbf{A}^{\mathrm{H}} = \begin{bmatrix} 1 & 1-i \\ -2+i & -i \\ 5 & 4+2i \end{bmatrix}$$

we can write $e^{jm\theta} = cos(m\theta) + jsin(m\theta)$ in

$$f_q(x_m, m) = e^{jm\theta} W_Q x_m$$

as a matrix as follows

$$e^{jm\theta} = \begin{bmatrix} cos(m\theta) & -sin(m\theta) \\ sin(m\theta) & cos(m\theta) \end{bmatrix}$$

Can you recognize this matrix?

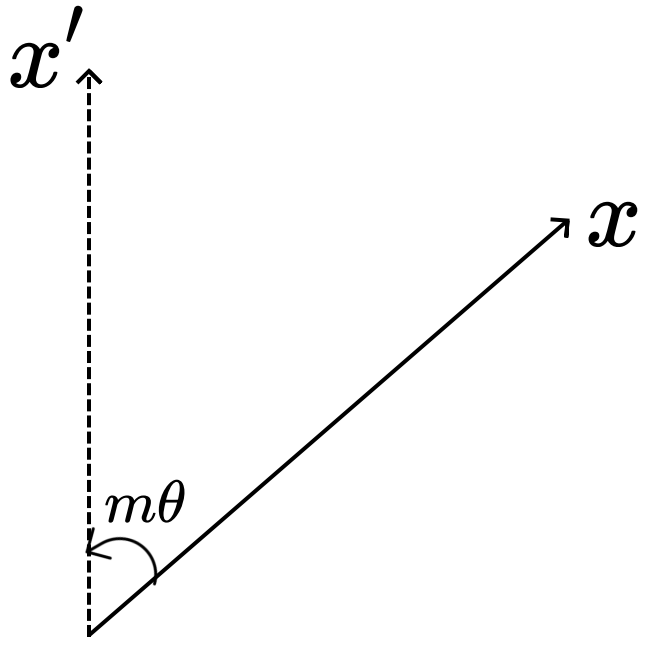
yeah, a rotation matrix $\mathbf{R}$

Therefore, for $x_m \in \mathbb{R}^2$

$$f_q(x_m, m) = \begin{bmatrix} cos(m\theta) & -sin(m\theta) \\ sin(m\theta) & cos(m\theta) \end{bmatrix} \begin{bmatrix} W_q^{11} & W_q^{12} \\ W_q^{21} & W_q^{22} \end{bmatrix} \begin{bmatrix} x_m^1 \\ x_m^2 \end{bmatrix}$$

$$f_q(x_m, m) = \mathbf{R} W_Q x_m$$

Essentially, it rotates affine-transformed x_m by $m\theta$



For efficient computation, we can write $e^{jm\theta} = cos(m\theta) + jsin(m\theta)$ in

$$f_q(x_m, m) = e^{jm\theta} W_Q x_m$$

as a matrix as follows

$$e^{jm\theta} = \begin{bmatrix} cos(m\theta) & -sin(m\theta) \\ sin(m\theta) & cos(m\theta) \end{bmatrix}$$

Can you recognize this matrix?

ya, a rotation matrix $\mathbf{R}$

Therefore, for $x_m \in \mathbb{R}^2$

$$f_q(x_m, m) = \begin{bmatrix} cos(m\theta) & -sin(m\theta) \\ sin(m\theta) & cos(m\theta) \end{bmatrix} \begin{bmatrix} W_q^{11} & W_q^{12} \\ W_q^{21} & W_q^{22} \end{bmatrix} \begin{bmatrix} x_m^1 \\ x_m^2 \end{bmatrix}$$

$$f_q(x_m, m) = \mathbf{R} W_Q x_m$$

similarly for $f_k(x_n, n)$

$$f_k(x_n, n) = \begin{bmatrix} cos(n\theta) & -sin(n\theta) \\ sin(n\theta) & cos(n\theta) \end{bmatrix} \begin{bmatrix} W_k^{11} & W_k^{12} \\ W_k^{21} & W_k^{22} \end{bmatrix} \begin{bmatrix} x_n^1 \\ x_n^2 \end{bmatrix}$$

$$f_k(x_n, n) = \mathbf{R} W_K x_n$$

Essentially, the tranformation rotates the embeddings x_m, x_n by $m\theta, n\theta$, respectively

Then the inner product results in

$$\langle f_q(x_m, m), f_k(x_n, n) \rangle$$

$$= x_m^T W_Q^T \begin{bmatrix} cos(m\theta) & sin(m\theta) \\ -sin(m\theta) & cos(m\theta) \end{bmatrix} \begin{bmatrix} cos(n\theta) & -sin(n\theta) \\ sin(n\theta) & cos(n\theta) \end{bmatrix} W_K x_n$$

$$q_m^T k_n = x_m^T W_Q^T \begin{bmatrix} cos((m-n)\theta) & sin((m-n)\theta) \\ -sin((m-n)\theta) & cos((m-n)\theta) \end{bmatrix} W_K x_n$$

We can generalize this to $x \in \mathbb{R}^d, d > 2$

As show in the figure, we can extend the idea to $x_m \in \mathbb{R}^d$ as follows

$$f_q(x_m, m) = \mathbf{R}^d_{\Theta,m} W_Q x_m$$

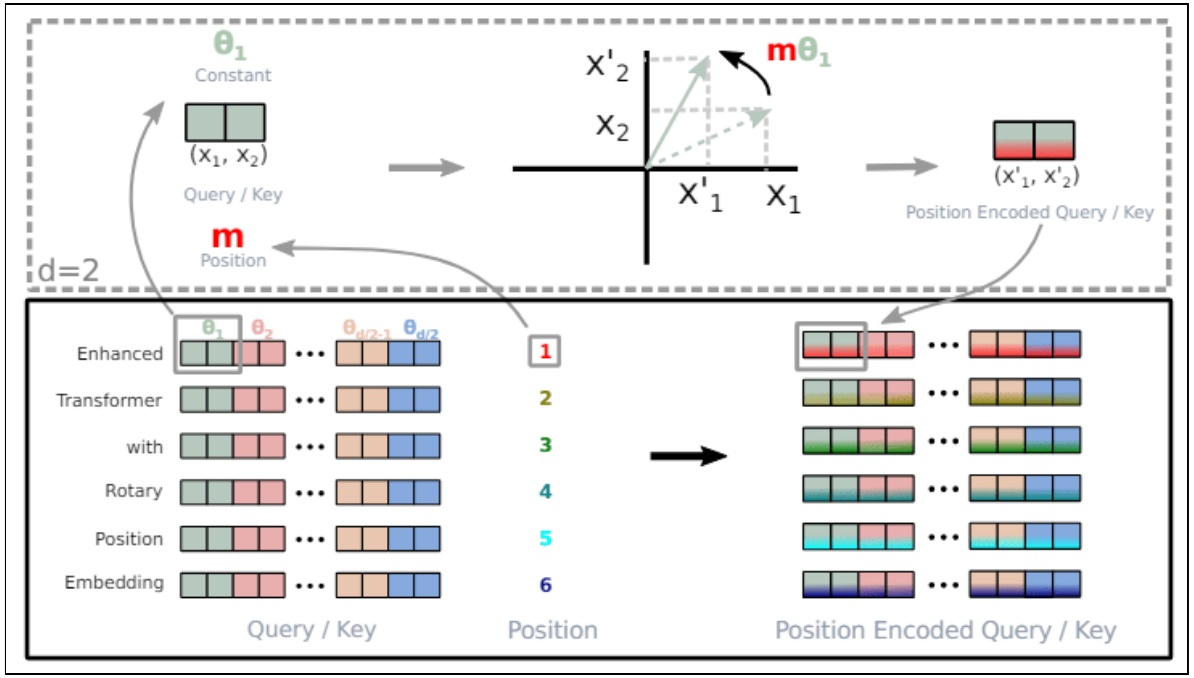
where $\mathbf{R}^d_{\Theta,m}$ is given by

$$\mathbf{R}^d_{\Theta,m} = \begin{bmatrix} cos(m\theta_1) & -sin(m\theta_1) & 0 & 0 & \cdots & 0 & 0 \\ sin(m\theta_1) & cos(m\theta_1) & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & cos(m\theta_2) & -sin(m\theta_2) & \cdots & 0 & 0 \\ 0 & 0 & sin(m\theta_2) & cos(m\theta_2) & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & cos(m\theta_{d/2}) & -sin(m\theta_{d/2}) \\ 0 & 0 & 0 & 0 & \cdots & sin(m\theta_{d/2}) & cos(m\theta_{d/2}) \end{bmatrix}$$

$$\Theta = \{\theta_i = 10000^{-\frac{2(i-1)}{d}}, i \in (1, 2, \cdots, d/2)\}$$

It is similar to the one we used in sinusoidal embedding

Then, the pre-attention score is computed by encoding the relative position as follows



$$q_m^T k_n = (\mathbf{R}^d_{\Theta,m} W_Q x_m)^T (\mathbf{R}^d_{\Theta,n} W_K x_n) \longleftarrow \text{APE of } m, n$$

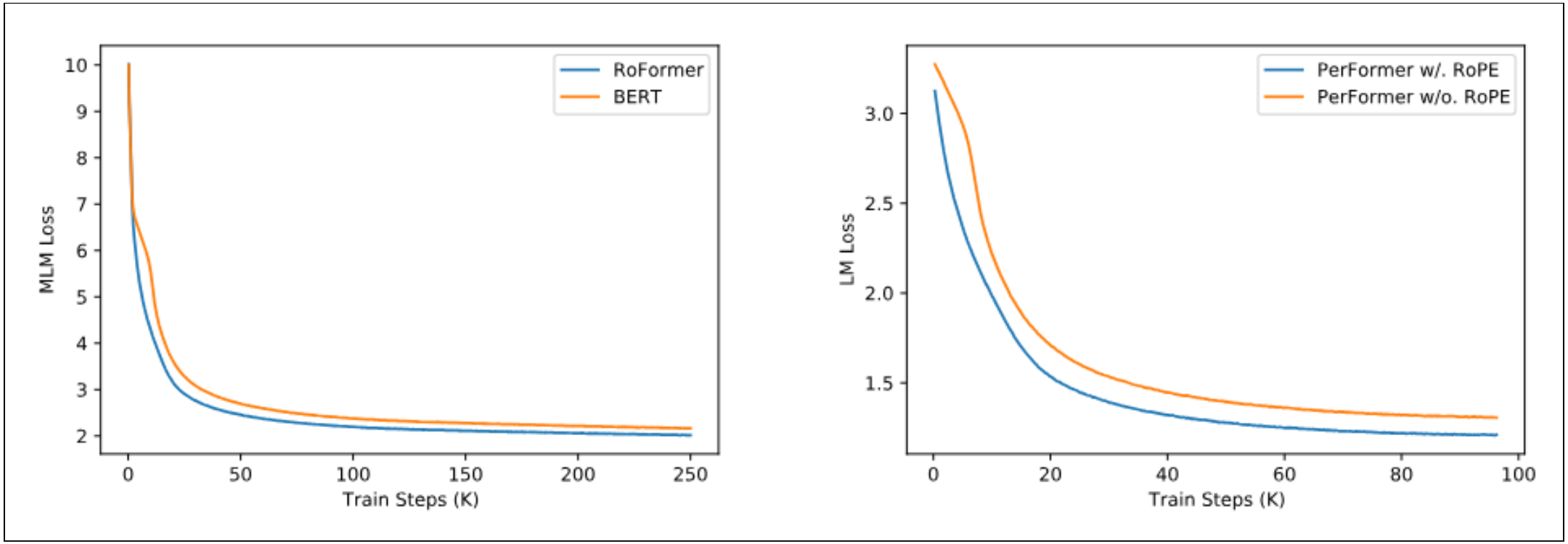
$$= x_m^T W_Q^T (\mathbf{R}^d_{\Theta,m})^T (\mathbf{R}^d_{\Theta,n}) W_K x_n$$

$$= x_m^T W_Q^T (\mathbf{R}^d_{\Theta,m})^T (\mathbf{R}^d_{\Theta,n}) W_K x_n$$

$$\boxed{= x_m^T W_Q^T \mathbf{R}^d_{\Theta,m-n} W_K x_n} \longleftarrow \text{RPE of } m - n$$

$$\text{where,} \mathbf{R}^T_{\Theta,n-m} = (\mathbf{R}^d_{\Theta,m})^T (\mathbf{R}^d_{\Theta,n})$$

Performance of RoPE during pre-training



Unlike sinusoidal APE, RoPE injects the positional information at every layer of the model and the position embedding is not injected into the value vectors.

Module 3 : PE and Length Generalization

Mitesh M. Khapra

AI4Bharat, Department of Computer Science and Engineering, IIT Madras

So far, we talked about various PE methods which lead to better performance

How do they help in length generalization?

Let's define length generalization as the ability of a model to extrapolate to longer sequences than the one it has seen during training

From the transformer paper

> We also experimented with using learned positional embeddings [9] instead, and found that the two versions produced nearly identical results (see Table 3 row (E)). We chose the sinusoidal version because it may allow the model to extrapolate to sequence lengths longer than the ones encountered during training.

However, no experiments were conducted to validate the same

Which approach (APE or RPE) is good at extrapolation?

Attention with Linear Bias (ALiBi)

Suppose we train a causal language model with $T = 512$ tokens

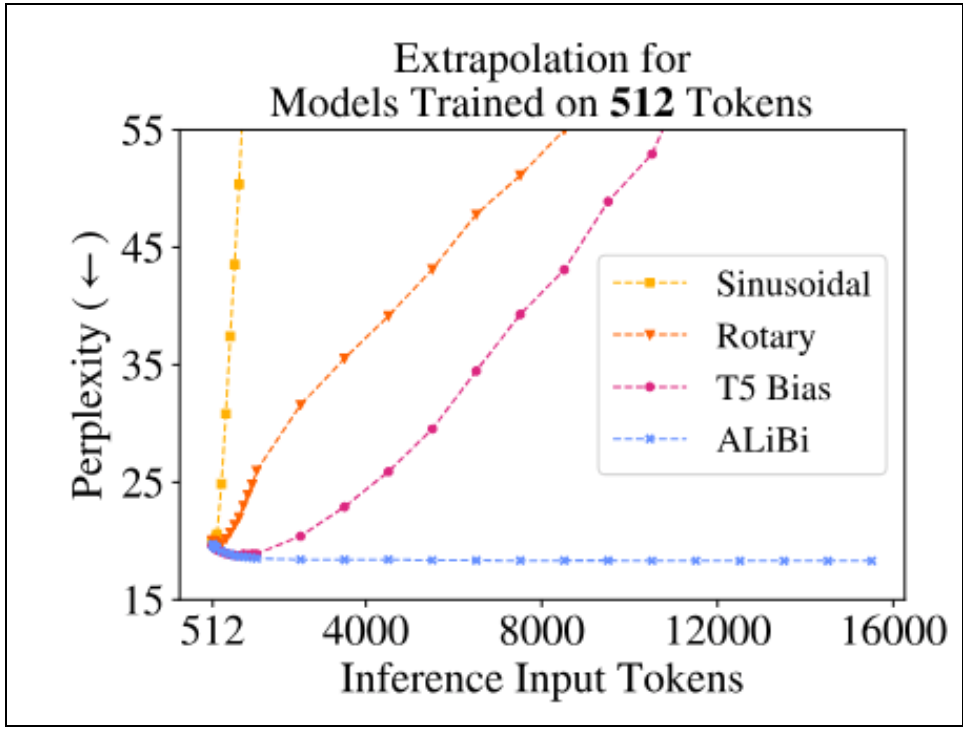
During inference, we validate the model by allowing the model to predict next tokens up to T_{valid} tokens

Perplexity score is used to measure the quality of predictions.

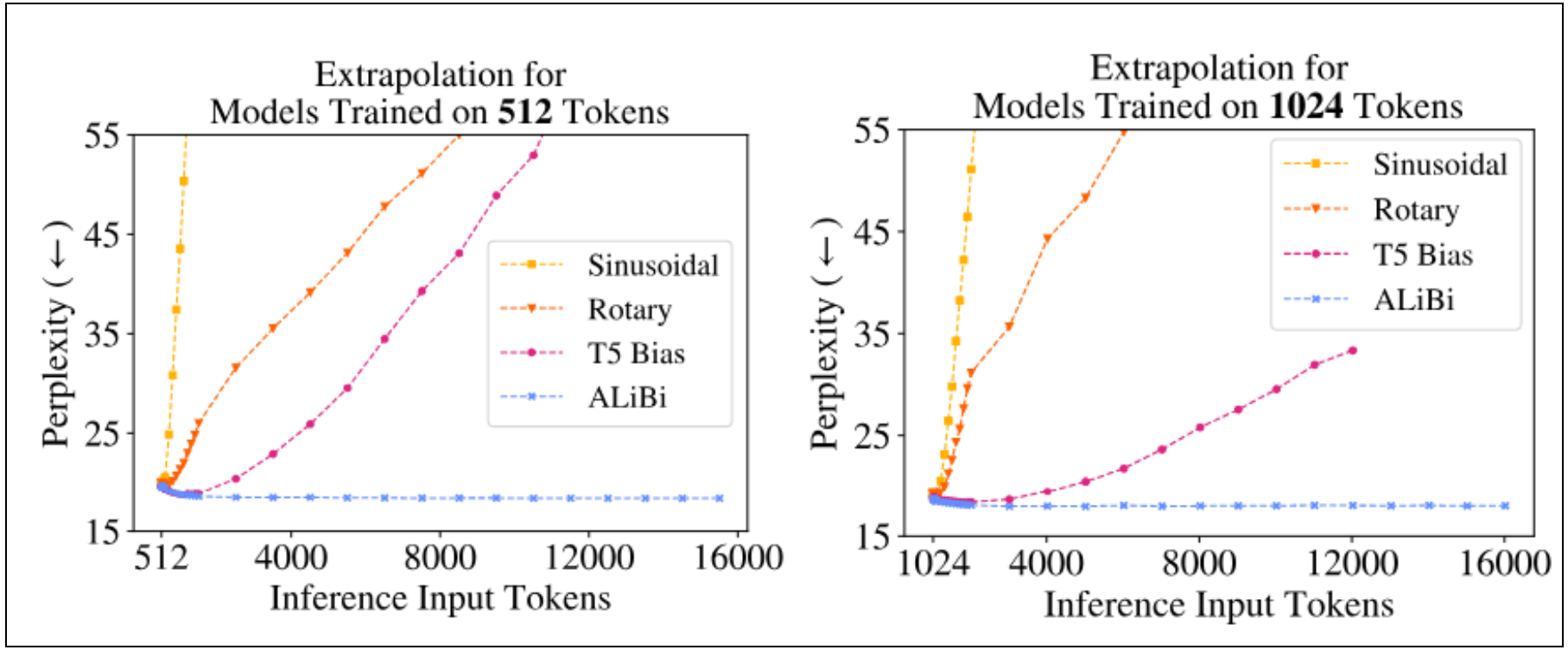
T_{valid} is varied from 512 to 16000 tokens

We repeat the experiment only by changing the position encoding method and keeping the other aspects as is.

The figure below shows the performance of using sinusoidal, T5, RoPE and ALiBi position encoding methods



Does increasing the training context window $T = 1024$ help?



Does increasing the training context window $T = 1024$ help?

Yes, it helps T5 Bias RPE more than rotary and sinusoidal APE. For example, for $T_{valid} = 8000$, the model trained with $T = 512$ has a higher perplexity score than the model trained with $T = 1024$

This suggests that modifying T5-bias might improve the performance.

Attention with Linear Bias (ALiBi)

Recall the T5-bias RPE

$$e_{ij} = x_i W_Q W_K^T x_j^T + r_{j-i}$$

where

$r_{j-i} \in \mathbb{R}$ are learnable scalars

Why not hard-code r_{j-i}?

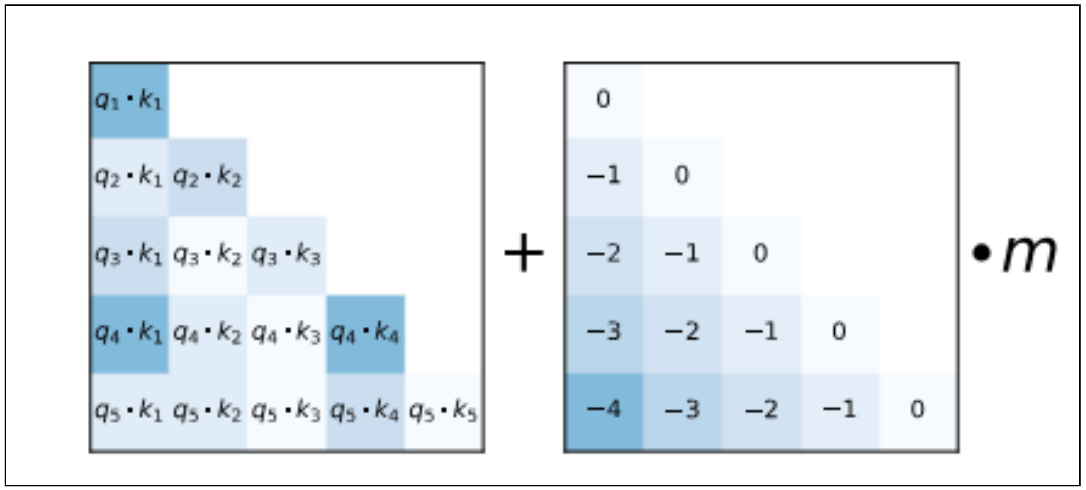
as it helps increase the inference speed

How do we do that?

Just add a constant (negative bias) to the attention score based on its position as show in the figure

The attention score is computed as follows

$$\text{softmax}(q_i K^T + m.(-(i-1), \cdots, -2, -1, 0)))$$



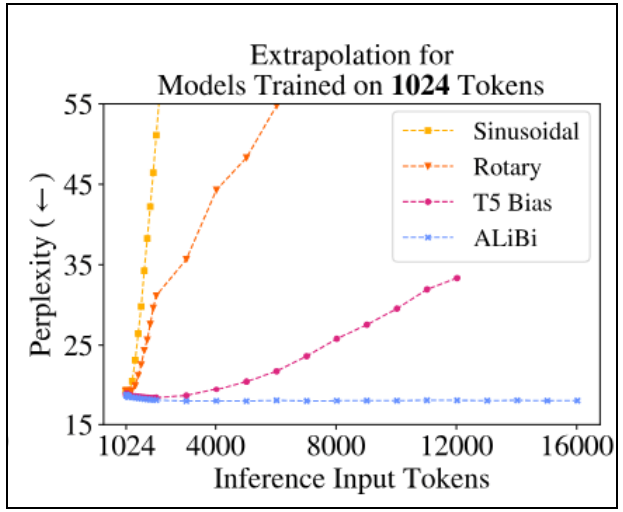
m is a head specific scalar that follows geometric progression.

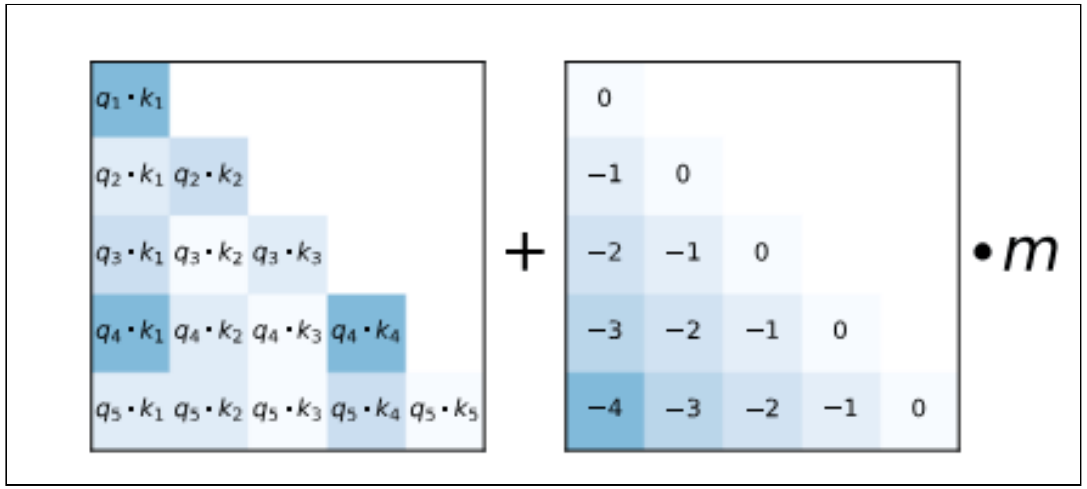
For $n = 8$ heads, $m_i = \frac{1}{2^i}, i = 1, \cdots, n$

Since it is RPE, position information is added at every layer of the model.

It is called linear because the bias value increase linearly with respect to the distance

ALiBi Acts Like a Local Windowed Attention





Why does this work remarkably well even for $T_{valid} = 16K$?

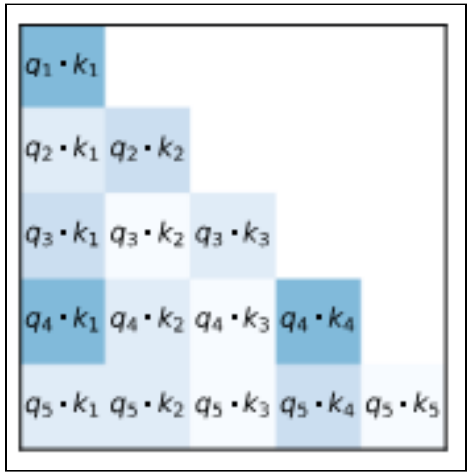
We can observe that the constant value added for nearby tokens is larger (0,-1,-2..) than the one added for distant tokens say (-128,-129..)

This effectively acts as a local windowed attention that enables the ALiBi PE to predict the next token effectively from the recent past k tokens (a natural structure in languages)

Subsequent studies generalized ALiBi such as KERPLE (Kernalized Relative Position embedding for Length Extrapolation) and improved RoPE such as xPOS.

Is that all? "NoPE"

A study [NoPos] conjectures that using a causal mask **implicitly** encodes the position information



	WikiText-103	The Pile
NoPos	20.97	13.10
Learned	20.42	13.05
Sinusoidal	20.16	12.93
ALiBi	19.71	12.51

Validation perplexity

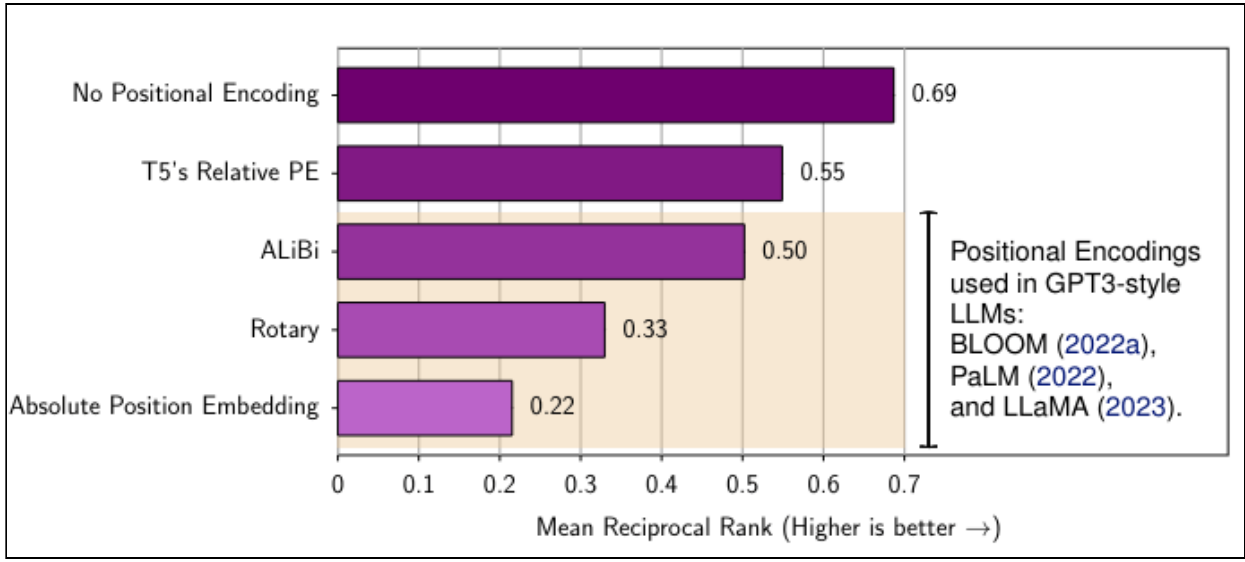
The previous approach (ALiBi) used "perplexity" as a metric to measure the length generalization ability during inference

This may not be suitable for all downstream tasks like sorting, addition, summation,reversing...

Keeping these two observations in mind, one could extend the study on length generalization ability of decoder-only models, to other downstream tasks

One study considered sinusoidal APE, RoPE, T5-bias, No Position Encoding (NoPE)

Here is the summary of the findings



Is that all? "NoPE"

A study [NoPos] conjectures that using a causal mask implicitly encodes the position information

It is shown theoretically that NoPE (i.e, decoder-only models)

1. can represent both absolute and relative PEs
2. learns to use relative PE in practice

Despite all these studies, length generalization remains a challenge for transformer based models

This raises a question

Is there an inherent limitation in Transformers' design preventing effective length generalization?

a recent study (Feb 2024) suggests so

Google DeepMind

Transformers Can Achieve Length Generalization But Not Robustly

Summary

	PE	Manifestation	Learnable	Integration	Injection Layer
APE	Sinusoidal (Vaswani et al., 2017)	Embedding	✖	Add	Initial
	SHAPE (Kiyono et al., 2021)	Embedding	✖	Add	Initial
	CAPE (Likhomanenko et al., 2021)	Embedding	✖	Add	Initial
	Complex (Wang et al., 2021)	Embedding	✔	Multiply	Initial
	FLOATER (Liu et al., 2020)	Embedding	✔	Add	Initial
RPE	Shaw et al. (2018)	Embedding	✔	Add	Every
	T5 Bias (Raffel et al., 2020)	Bias	✔	Add	Every
	ALiBi (Press et al., 2022)	Bias	✖	Add	Every
	KERPLE (Chi et al., 2022)	Bias	✔	Add	Every
	SANDWICH (Chi et al., 2023b)	Embedding	✖	Add	Every
	FIRE (Li et al., 2023)	Bias	✔	Add	Every
	Transformer-XL (Dai et al., 2019)	Embedding	✖	Add	Every
	RoPE (Su et al., 2024)	Embedding	✖	Multiply	Every
	xPOS (Sun et al., 2023)	Embedding	✖	Multiply	Every

NoPE is not in the list for obvious reasons (but we can list it under RPE)

source:[paper]